



P3

# **The Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate: A Pre-Registered Full-Factorial Randomized Controlled Simulation**

故障熔断治理对 Agent 矩阵收口率的因果效应

---

# The Causal Effect of Fault-Breaker Governance on Agent Matrix Completion Rate: A Pre-Registered Full-Factorial Randomized Controlled Simulation

---

This is a causal-inference paper of the UDOS Reasoning Engine v7.5.0 (main tag v7.5.0, commit 1a71270). The governance trio (circuit breaking / snapshot rollback / reassignment), Owner/Trace/Stop governance, and the matrix end-to-end loop are all runnable implementations in this repo's `udos7/topology/`. The current report is a **single-seed pilot point estimate**; the design, primary metric, guardrails, and stopping rules of the full-factorial randomized controlled simulation have been written under pre-registration discipline, and the multi-seed experiment is pending before submission (see Sec. 5 and 8). All numbers come from `reports7/matrix_scale_demo.json` (evidence grade cpu-proto).

## Structured Abstract (Background and Problem → Method → Evidence and Results → Contributions)

**[Background and Problem]** Multi-agent matrices inevitably encounter faults (crashed executors, handoff context loss, Byzantine erroneous outputs). Industry demos mostly report only the "fixed" success cases; they neither decompose "which governance mechanism is actually acting," nor report "whether the system honestly fails when it cannot fix it," nor report the cost of governance itself (see § 1.1, § 2.4).

**[Method]** We decompose fault governance into independently switchable factors: the resilience trio (three-tier circuit breaking / snapshot rollback / reassignment) and responsibility governance (unique Owner / hash-chained Trace / explicit Stop Condition), and use a **pre-registered full-factorial randomized controlled simulation** to estimate the causal effect of "governance switches × fault modes" on completion rate; the primary metric completion rate = accepted/total (the fraction of work orders mechanically accepted by QA), and the guardrails are reworks, messages, false-isolation rate, Trace breakage, and spurious kill-switch triggers (see § 3, § 4).

**[Evidence and Results]** Single-seed pilot (24 work orders, 9 specialists, 7 QA including 2 Byzantine): no faults give 24/24 completion and 0 reworks; after injecting crash/drop\_context/byzantine faults it still gives 24/24 completion, all 3 faulty nodes are isolated, 5 reworks occur, and governance, conservation, and Trace integrity are maintained; under the extreme "all-domain fault + zero retry budget" scenario, the system honestly trips the kill-switch and reports non-completion (accepted<24, governance.ok=false) rather than faking success (see § 6).

**[Contributions]** (i) Factorizing governance mechanisms into independently switchable, measurable marginal contributions; (ii) under pre-registered RCT discipline (freezing primary metric/guardrails/ITT stopping rules/seed table), fixing the causal design and adding data later; (iii) treating "honest failure (kill-switch)" as a first-class result. The current report is a single-seed pilot point estimate; the full factorial  $(5 \times 5 \times 5 \times 4) \times \geq 30$  seeds and the statistics are pending, and this paper does not fabricate p-values/CIs (see § 7, § 8).

---

## Chinese Abstract

多智能体（multi-agent）矩阵在真实运行中必然遇到故障：执行节点崩溃、交接丢失上下文、甚至给出拜占庭式错误输出。问题不在于“会不会坏”，而在于“治理机制能否因果地把收口率拉回来，代价是什么”。本文把故障治理拆成三组可独立开关的机制——熔断（三级断路器）、快照回滚（哈希链 Trace 截断）、改派（在途工单转交健康节点），以及一组责任治理（唯一 Owner、哈希链 Trace、显式 Stop Condition）——并以预注册的全因子随机对照仿真，估计“治理开关 × 故障模式”对任务收口率的因果效应。本文先在 24 工单、9 名 specialist、7 名 QA（含 2 名拜占庭 QA）的小规模矩阵上给出单 seed 试点：无故障时 24/24 收口、0 返工；注入 3 个故障 specialist（crash / drop\_context / byzantine 各一）后，仍 24/24 收口、3 个故障节点全部隔离、5 次返工、治理六项与守恒、Trace 完整性保持；而在“全领域故障 + 零重试预算”的极端场景下，系统如实触发 kill-switch、报告未收口（accepted<24、governance.ok=false），不冒充成功。主指标预注册为收口率（accepted/total），护栏为返工次数、消息总数、误隔离率、Trace 断裂数、kill-switch 误触发率。本文的贡献是：把故障治理因子化、预注册化，并诚实地把“能拉回收口率的机制”与“拉不回来、应如实失败的边界”同时测量出来。需要说明：上述“全领域故障 + 零重试预算触发 kill-switch”的反例为按设计构造的极端场景，当前仅在 one-pager/CHANGELOG 中定性记录，其精确 accepted 数与治理项明细尚待重跑确认（正文 6.4 节标 [RESULT NEEDED]），不在本次 matrix\_scale\_demo.json 的 clean/stressed 两跑之内。

**Keywords (Chinese):** 多智能体系统；故障容错；熔断；随机对照实验；预注册；收口率；拜占庭容错

---

## English Abstract

Multi-agent matrices inevitably encounter faults: crashed executors, lost handoff context, and even Byzantine outputs. The question is not whether faults occur, but whether governance mechanisms causally recover task completion, and at what cost. This paper decomposes fault governance into independently switchable mechanisms—three-tier circuit breaking, snapshot rollback over a hash-chained Trace, and reassignment—and a responsibility layer (unique Owner, hash-chained Trace, explicit Stop Condition). A pre-registered full-factorial randomized controlled simulation estimates the causal effect of "governance switches × fault modes" on completion rate. We first report a single-seed pilot on a 24-order matrix (9 specialists, 7 QA voters including 2 Byzantine): no faults yield 24/24 completion with 0 reworks; injecting three faulty specialists (crash / drop\_context / byzantine) still yields 24/24 completion, isolates all three faulty nodes, incurs 5 reworks, and keeps governance, conservation, and Trace integrity; under an extreme "all-domain fault plus zero retry budget" scenario, the system honestly trips the kill-switch and reports non-completion (accepted<24, governance.ok=false) rather than faking success. The pre-registered primary metric is completion rate; guardrails are reworks, message count, false-isolation rate, Trace breakage, and spurious kill-switch triggers. We contribute a factorized, pre-registered measurement of fault governance that reports both what recovers completion and the boundary beyond which honest failure is the correct behavior. Note: the "all-domain fault plus zero retry budget trips the kill-switch" counterexample is a designed extreme scenario currently recorded only qualitatively in a one-pager/CHANGELOG; its exact accepted count and governance details are pending a re-run (see Sec. 6.4), and it is not part of the clean/stressed runs in the current matrix\_scale\_demo.json.

**Keywords:** multi-agent systems; fault tolerance; circuit breaker; randomized controlled trial; pre-registration; completion rate; Byzantine fault tolerance

---

# | 1 Introduction

## 1.1 External Industry Motivation (all external figures are unverified, report-based, not independently verified)

As multi-agent orchestration systems move into production, "fault governance" changes from an engineering supporting role into a core problem. External reports are full of narratives such as "multi-agent autonomously completes complex workflows" and "self-healing Agent systems" (such external claims are **report-based, not independently verified**, marked `unverified`, used only as motivation, and not cross-validated with this repo's numbers). [CITATION NEEDED: LLM agent fault tolerance self-healing multi-agent production] But what the industry generally lacks is an experiment that turns "to what degree governance is causally effective" into a layered, switchable, pre-registered measurement. Most demos only show the "fixed" success cases; they do not report "whether the system honestly fails when it cannot fix it," nor do they report "the cost of governance itself" (falsely isolating healthy nodes, extra reworks, extra messages).

This paper argues that an honest fault-governance study must answer three questions at once: (1) Which governance mechanisms causally raise the completion rate? (2) What are the guardrail costs of these mechanisms? (3) Under what extreme scenarios is the correct behavior not "forcing a fix" but "honest failure"?

The reason "honest failure" is raised separately is that multi-agent systems have a natural temptation: when the system cannot complete the task, it can still output a result that "looks complete"—perhaps some Agent fabricated an answer, perhaps the QA committee was captured by a Byzantine majority, perhaps the scheduler forced a completion so as not to disappoint the user. In production systems, this "false success" is far more dangerous than "openly stating failure": it silently passes the error downstream, and by the time it erupts it can no longer be traced. Therefore, this paper defines the goal of the governance system as three things: **recover when recovery is possible, isolate and reassign when recovery is not possible, and honestly stop and leave a complete audit trail when it is utterly impossible**. These three goals are all indispensable; doing only the first two and omitting the third is not an honest governance system.

External industry narratives (such as "self-healing Agent" and "autonomous diagnostic workflow," all unverified report-based claims) mostly demo only the success cases of the first two goals. What this paper adds is the measurement of the third goal, and the marginal causal contribution of each mechanism in the first two goals.

Going further, the current literature on multi-agent fault governance has two common shortcomings: first, "black-box effectiveness"—it only reports how much the success rate improves after governance is turned on, without decomposing whether circuit breaking, rollback, or reassignment is acting; second, "reporting good news and hiding bad news"—it only shows recovery success cases and avoids "when the system should admit it cannot do it." The first leaves engineers unable to know which mechanism to invest in; the second makes production systems output untrustworthy results when recovery capacity is exceeded. This paper addresses the first shortcoming with factorized switches and the second with the honest reporting of the kill-switch counterexample. This is exactly why this paper is positioned as a "causal effect" rather than an "effect demo."

## 1.2 Research Questions

This paper focuses on three research questions:

- **RQ1:** What are the marginal contributions of the unique Owner, hash-chained Trace, and explicit Stop Condition? Removing which one leads to what type of failure?
- **RQ2:** Under crash / drop\_context faults, can the circuit-breaking/rollback/reassignment trio bring the completion rate back to the no-fault level? When the threshold is too low, will it falsely isolate healthy nodes (guardrail degradation)?
- **RQ3:** Can byzantine faults be isolated by executor-layer circuit breaking alone? Or must they be isolated through QA consensus plus accumulated quality failures?

## 1.3 Contributions

- **Factorized governance switches:** Decomposing governance into independently closable mechanisms ( -Owner , -Trace , -Stop , -circuit-breaking , -rollback , -reassignment ), so marginal contributions can be experimentally separated.
- **Pre-registered RCT design:** Under randomized controlled trial discipline, freezing in advance the primary metric, guardrails, stopping rules, randomization unit, and inclusion/exclusion criteria (Sec. 4).
- **Pilot evidence including "honest failure":** Reporting not only the 24/24 completion success case but also the counterexample in which, under "all-domain fault + zero retry budget," the kill-switch trips and the system honestly reports non-completion.
- **Honest labeling:** The current report is a single-seed pilot point estimate; the full-factorial multi-seed experiment is pending, and this paper does not fabricate p-values, CIs, or sample sizes.

## 1.4 Scope Statement

This paper studies "the causal effect of fault governance mechanisms on completion rate," not "how to train a more reliable LLM." The Agent's intelligence is provided by external models, which this paper abstracts as deterministic processors, and faults are injected manually. Therefore, the conclusions are causal effects at the protocol and mechanism layers, and do not involve improving the reliability of the models themselves. The intended readers are systems researchers working on multi-agent system design, fault tolerance, and pre-registered causal experiments. Conclusions beyond this boundary (real model reliability, real network latency) are marked as not reached in the gate of Sec. 8.

---

## 2 Related Work

### 2.1 Microservice Circuit Breaking and Fault Tolerance

Circuit breaking is mature in microservice governance: closed/open/half-open state machines, error-rate thresholds, cooldown probes, half-open recovery. [CITATION NEEDED: circuit breaker pattern microservices fault tolerance] Another thread, sharing the same origin as "degrading under uncertainty," is conformal prediction: at the **concept** level, when a prediction is uncertain one should give a coverage guarantee rather than forcing a conclusion; at the **mechanism** level, the conformal prediction survey by Angelopoulos and Bates provides distribution-free, finite-sample prediction intervals (Angelopoulos & Bates, 2021, arXiv:2107.07511); at the **evidence** level, it shows that "degrade/handoff when uncertainty is high" is a design principle with a statistical foundation. This repo's `udos7/topology/circuit_breaker.py` extends the circuit-breaking pattern to three tiers: an Agent-level breaker, a sub-matrix group breaker,

and a global kill-switch, and adds the multi-agent-specific recovery action of "reassigning in-flight work orders on fault."

Microservice circuit breaking usually acts only on remote-call failures, whereas faults in multi-agent scenarios are more "semantic": an Agent may, without any network error, give a result that looks successful but is actually wrong (Byzantine), or be unable to continue because context was lost during handoff (drop\_context). Such faults do not trigger traditional network timeouts or 5xx, so the definition of "error" must be extended from "call failure" to "the result does not satisfy the acceptance predicate." On the embodied side, SayCan echoes this: at the **concept** level, the LLM gives step candidates and the skill value function gives feasibility affordance; at the **mechanism** level, when affordance is not satisfied the step is not executed (Ahn, Brohan, Brown et al., 2022, arXiv:2204.01691); this paper generalizes "switch person/skill when affordance is not satisfied" to "trigger circuit breaking/reassignment when the acceptance predicate is not satisfied." The governance experiment of this paper is precisely meant to measure: under such semantic faults, can the traditional circuit-breaking pattern plus "result-acceptance-driven accumulation of quality failures" causally recover the completion rate.

## 2.2 Byzantine Fault Tolerance and Stop Consensus

BFT consensus is a mature mechanism: at the **concept** level, how to still agree on state when arbitrarily malicious (equivocating) nodes exist in an asynchronous network; at the **mechanism** level, Castro and Liskov's PBFT uses three-phase replication of pre-prepare/prepare/commit, requires  $n \geq 3f+1$  to tolerate  $f$  Byzantine nodes and a  $2f+1$  quorum, together with view change (Castro & Liskov, 1999, OSDI 1999); at the **evidence** level, its BFS implementation is only about 3% slower than unreplicated NFS. This repo's `udos7/topology/consensus.py` `StopConsensus` borrows it to vote on "acceptance (stop) / rework (continue)": a Byzantine QA's false report cannot defeat the honest majority. Another thread sharing the same origin as the QA committee is LLM-as-a-Judge: Zheng et al. show that strong models as reviewers have >80% agreement with human judges but exhibit position/verbosity/self-preference biases (Zheng, Chiang, Sheng et al., 2023, NeurIPS 2023 Datasets, arXiv:2306.05685); this suggests that "voting with multiple QA" itself needs protection against judge bias, and this pilot uses a 7-member committee precisely to suppress single-point bias with majority vote.

In this pilot, the QA committee has 7 members, 2 of whom are Byzantine ( $f=2$ ), satisfying the strictest Byzantine fault-tolerance condition  $n \geq 3f+1$  ( $7 \geq 7$ ). This means that even if 2 QA collude to vote against, the honest majority can still correctly decide acceptance. The presence of 2 Byzantine QA in this pilot is precisely meant to test whether "acceptance-layer consensus can still complete correctly in the presence of Byzantine nodes"—the result is 24/24 completion, showing the consensus design works as intended. It must be emphasized that this is only a demonstration of protocol-layer correctness, not a measurement of real LLM voting behavior.

## 2.3 Observability and Responsibility Tracing

Hash-chained append-only logs can detect tampering and broken links; unique Owner registration can detect duplicate work. [CITATION NEEDED: hash chained audit log ownership attribution] This repo's `governance.py` `TraceLedger` + `audit_run` mechanize the detection of five failure classes: state loss, duplicate work, unclear responsibility, premature termination, and unbounded loops.

Unlike ordinary observability that only "displays metrics," the responsibility governance of this paper makes "who is responsible for which stage" and "whether this result is traceable" into hard constraints: once `claim(order, stage, agent)` finds the same order:stage claimed by a second, different owner, it is counted as `duplicate_work`; `audit_run` gives an explicit list for each failure class at the end of a run. This

makes "duplicate work" and "unclear responsibility" no longer vague concepts requiring post-hoc manual review, but guardrail metrics directly countable in the experiment.

## 2.4 Existing Gaps

We hypothesize (pending retrieval to confirm/refute): circuit breaking is mature in microservices, but there is a lack of factorized causal experiments under Agent **semantic faults** (context loss, Byzantine outputs); few studies treat "governance honestly failing" as a first-class measurement goal. At the **concept** level, conversable multi-agent frameworks such as AutoGen have observed the multi-agent-specific failure mode of "conversations without termination conditions easily dead-loop, with cost inflating with the number of rounds" (Wu, Bansal, Zhang et al., 2023, arXiv:2308.08155); at the **mechanism** level, pure-RL approaches such as DeepSeek-R1 demonstrate the possibility of "recovery and self-play driven by verifiable rewards after circuit breaking/failure" (DeepSeek-AI, 2025, arXiv: 2501.12948). At the **evidence** level, these works prove that multi-agent failure and recovery are real problems, but they are mostly discussed at the framework/training level, with few doing controlled causal measurement with governance switches as factors. **The difference from UDOS** is: this paper decomposes governance mechanisms into a switchable trio, uses a pre-registered RCT to measure their causal effects, and treats "governance honestly failing" as a first-class measurement goal. [CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] This paper attempts to fill this gap with a pre-registered RCT.

## 2.5 Pre-registration and Reproducible Causal Inference

In systems and experimental research, "picking metrics after seeing results" is one of the most common validity threats. Pre-registration reduces this peeking bias by freezing the primary metric, guardrails, sample size, and stopping rules before collecting data. [CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] This paper applies this discipline to the fault governance experiment: the factors in Sec. 4.4, the primary metric (completion rate), the guardrails (reworks/messages/false isolation/trace breakage), and the ITT stopping rules are all frozen before running the full factorial. The single-seed pilot reported now is the first step of "design first, data later": fixing the plan first and then gradually adding data, rather than the reverse. This stance is consistent with the evidence-grading methodology of this repo's P12 paper.

## 2.6 Chaos Engineering and Fault Injection

Chaos engineering advocates actively injecting faults into the system and observing resilience rather than waiting for real incidents. [CITATION NEEDED: chaos engineering fault injection resilience] The fault injection of this paper (crash / drop\_context / byzantine / duplicate) is precisely the miniaturization of this idea onto a multi-agent matrix: rather than waiting for a real LLM to fail, it actively, controllably, and reproducibly injects semantic faults on the test bench and measures the response of governance mechanisms. The difference is that traditional chaos engineering mostly injects network latency and node crashes at the microservice/infrastructure layer, whereas this paper additionally injects "semantic faults"—nodes that look normal but produce wrong results (byzantine), which is a multi-agent-specific failure mode.

---

## 3 Problem Definition and Hypotheses

### 3.1 Fault Model

This paper uses four manually injected discrete faults (source `FAULTS` and `AgentSpec.fault`):

- **crash**: The executor fails directly (`error_type="crash"`), with no output.
- **drop\_context**: The executor loses context (`error_type="missing_context"`, `note="domain/units not received"`) and cannot complete correctly.
- **byzantine**: The executor "looks successful" but outputs a wrong result (the specialist returns `true_result + 7`, or triage states the domain incorrectly).
- **duplicate**: Duplicate work.

### 3.2 Falsifiable Hypotheses

- **H1 (marginal governance elements)**: Removing the unique Owner → duplicate-work rate rises; removing the Trace → after state loss there is no replay and completion rate falls; removing the Stop Condition → unbounded loops or premature termination appear.
- **H2 (trio recovery)**: Circuit breaking + rollback + reassignment bring the completion rate under crash / drop\_context back to the no-fault level; if the circuit-breaking threshold is too low, healthy nodes are falsely isolated (guardrail degradation).
- **H3 (Byzantine requires consensus)**: Byzantine faults cannot be isolated by executor-layer circuit breaking alone; they must be isolated through QA consensus plus accumulated quality failures.

**Falsifiability criteria**: If the completion rate does not change significantly after removing a governance element, the marginal-contribution hypothesis for that element is weakened; if crash faults still cannot complete under the trio, H2 is overturned; if byzantine faults can be isolated at the executor layer without QA consensus, H3 is weakened.

### 3.3 Why Use "Completion Rate" as the Primary Metric

The success of multi-agent tasks is hard to characterize with a single number, but the work orders in this paper are mechanically decidable: each work order hides a ground truth (domain + unit sum), and `WorkOrder.verify(result, domain)` strictly compares the result with the ground truth and the domain with the true domain. Therefore "completion" is operationalized as "the work order is ultimately accepted (stop) by the QA committee," and completion rate = accepted/total. This avoids the inter-rater inconsistency of subjective scoring and makes causal-effect estimation reproducible in a deterministic CPU environment. It must be emphasized that completion rate is chosen not because it is the only thing that matters, but because it is mechanically decidable on this test bench; rework count, message count, and false-isolation rate serve as guardrails, characterizing "the price paid for completion."

## 4 Method and System Design

### 4.1 The Trio: Three-Tier Circuit Breaking + Snapshot Rollback + Reassignment

Source `udos7/topology/circuit_breaker.py`:

- **AgentBreaker**: Sliding-window error rate; `record(ok)` accumulates success/failure, and when `successes+failures ≥ min_requests` and `error_rate ≥ trip_threshold` it goes OPEN (isolated); after a

cooldown of `cooldown_ticks` it goes `HALF_OPEN`, and only a successful `probe(ok)` returns it to closed. The `available` property is true only when closed, so broken nodes are excluded at dispatch.

- **GroupBreaker:** If the fraction of OPEN members in a group is  $\geq$  `isolate_ratio_threshold`, the whole group breaks.
- **KillSwitch:** After `trip(reason)`, `allow_dispatch()` is always false and all dispatch stops immediately.
- **snapshot / rollback:** Before dispatch, `snapshot(trace)` records the ledger length and last hash; after execution failure, `rollback` truncates the half-written Trace and reassigns to a healthy node. After rollback the hash chain is still verifiable.

## 4.2 Responsibility Governance: Owner / Trace / Stop

Source `udos7/topology/governance.py`:

- **TraceLedger:** Each record contains `prev`, the hash of the previous record, and this record's `h` (sha256 truncated to 16 chars); `verify()` returns the index of a broken link or a rewritten record.
- **audit\_run:** Mechanically detects five failure classes—`state_loss` (a failure record carries missing context or still has todos), `duplicate_work` (the same order:stage is claimed by two owners), `no_closer` (a work order finishes with no final completion), `premature_completion` (claims success but lacks a stage owner), `unbounded` (exceeds the hop budget).
- **StopCondition:** Completion predicate + `max_hops` hop budget, distinguishing "done" from "hop\_budget\_exceeded."

The meaning of the hash chain is worth expanding here. In fault governance, what is most frightening is not "the system made an error," but "the system made an error that cannot be traced." If the Trace is rolled back after execution failure, how does an auditor know it ever failed? This system's approach is: snapshot before dispatch (recording length and last hash), roll back to the snapshot point after failure—this means the half-written Trace is deleted, but **the circuit-breaking record and governance audit are independently retained**; the hash chain after rollback extends again from the snapshot point, and `verify()` still passes. In other words, "leaving no half-written state" and "being auditable" are not contradictory: the half-written state is removed from the main execution chain, but the failure event is separately recorded through `grid.record` and `audit_run`. In the pilot this design shows up as the Trace length changing from 48 in the clean run to 50 in the fault run (rework items re-enter the chain), with both `trace_verified` being true.

## 4.3 Matrix End-to-End Loop

Source `udos7/topology/matrix.py` `run_mission`: blackboard dispatch → broken-node exclusion → execution → failure rollback and reassignment → BFT-lite QA voting → market settlement → governance audit. Configuration (`MatrixConfig`): `specialists_per_domain=3` (9 specialists total), `qa_voters=7`, `qa_f=2`, `reward_per_order=10`, `max_rounds=20`, `breaker_min_requests=2`, `breaker_threshold=0.5`, `global_retry_budget=999`.

## 4.4 Pre-Registered Experiment Design (Full-Factorial RCT)

The following is the pre-registered plan to be executed before submission; currently only the single-seed pilot of the "full governance × fault injection" cell is complete.

- **Factor A governance:** {full, -Owner, -Trace, -Stop, all off} (5 levels).
- **Factor B resilience:** {full, -circuit-breaking, -rollback, -reassignment, all off} (5 levels).
- **Factor C fault type:** {crash, drop\_context, byzantine, duplicate, mixed}.

- **Factor D fault rate:** {0.05, 0.1, 0.2, 0.33}.
- **Randomization unit:** Independent seeds; faulty-node positions are randomized in each cell;  $\geq 30$  independent seeds per cell, with the seed table pre-registered.
- **Primary metric:** Completion rate accepted/total.
- **Guardrails:** Rework count, total messages, false-isolation rate (fraction of healthy nodes broken), Trace tampering detection rate, spurious kill-switch trigger rate, wall clock (CPU).
- **Stopping rules:** (a) The simulator has a fixed budget and is not extended by intermediate results; (b) cells where the kill-switch trips are retained as failures under ITT (intention-to-treat) and not removed because "it got fixed"; (c) no selective deletion of seeds; (d) the main contrast = full governance vs all off  $\times$  mixed faults.
- **Analysis plan:** Logistic mixed model (seed random intercept), reporting OR and 95% CI; Holm correction for multiple comparisons; effect size reported as  $\eta^2$ . Power analysis: after estimating variance from a 5-seed pilot, compute the required number of seeds [RESULT NEEDED: usage=number of seeds required for power analysis].

## 4.5 Breaker State-Machine Pseudocode

For reproducibility, here is the core state machine of the Agent-level breaker (corresponding to `AgentBreaker` in `circuit_breaker.py`):

```
States: closed -> open -> half_open -> closed
record(ok, tick):
    if state == open: return          # already isolated, no longer counted
    update successes/failures
    if (successes+failures >= min_requests) and (error_rate >= threshold):
        state = open; tripped_at = tick
tick(tick):
    if state==open and tick-tripped_at >= cooldown: state = half_open
probe(ok):
    # probe once during half-open
    state = closed if ok else open; reset counters
available(): state == closed          # only closed participates in dispatch
```

This pilot's parameters: `min_requests=2`, `threshold=0.5`, `cooldown_ticks=3`. This means a specialist is isolated once it has accumulated at least 2 calls with an error rate of 50%—a fairly sensitive threshold meant to isolate faults quickly; whether it falsely isolates healthy nodes is exactly what the guardrail "false-isolation rate" is meant to measure.

## 4.6 Fault-Loop Pseudocode

The core loop of `run_mission` (corresponding to `matrix.py`):

```

for round in range(max_rounds):
    for a in healthy_specialists(breakers excluded):
        claim a domain-matching work order on the blackboard
        snap = snapshot(trace)           # record length + last hash
        rep = execute(a, order)
        if not rep.ok:
            rollback(trace, snap)         # truncate the half-written Trace
            record the fault, release the work order for reassignment, retries++
            if retries > global_retry_budget: kill.trip(...)
            continue
        QA committee votes (stop/continue) # n>=3f+1, Byzantine votes go against
        if stop: completion + market settlement
        else: record a quality failure, work order re-hung on blackboard as rework, retries++
    if kill.engaged: break
governance audit (audit_run) + conservation check + Trace.verify

```

Key design: on failure, first roll back the Trace and then reassign, ensuring "no half-written state remains"; the QA committee uses BFT-lite consensus so 2 Byzantine QA cannot overturn the honest majority.

## 4.7 Governance Decision Tree for High-Risk Scenarios

Governance is not "the more aggressive the better." The `choose_topology` in source `udos7/topology/decision.py` gives a simple but important principle: when a task is "open exploration and high risk," the system should not let the Swarm self-govern, but should first add Orchestrator guardrails/circuit breaking and then release it (returning `escalate`, `autonomy_level=0`). This is continuous with this paper's fault governance: high-risk tasks need stricter governance switches (lower breaker thresholds, tighter retry budgets, stronger audits). In this paper's full-factorial design, the "fault rate" factor D is precisely used to approximate "risk exposure level"—the higher the fault rate, the closer to a high-risk scenario needing escalation. The decision tree encodes the engineering intuition that "governance intensity should adjust with risk" into experimentally measurable switches.

## 4.8 The Internal Market as a Settlement Guardrail

Fault governance has another often-overlooked dimension: economic incentives. The `ContributionLedger` in source `udos7/topology/market.py` pays only "the unique complecer whose work is accepted"; duplicate work is not paid, and Byzantine/rejected results are not paid and may have their deposit slashed. In the pilot, both clean and fault runs satisfy conservation (budget 240 = payout 240, `conserved=true`). This means governance not only isolates faulty nodes at the behavioral level but also ensures at the incentive level that "doing things recklessly gets no reward." This paper treats conservation as a process metric rather than the primary metric.

**Table 1 Full-factorial RCT factor design (pre-registered, pending execution)**

| Factor       | Levels                                                         | Notes                                                         |
|--------------|----------------------------------------------------------------|---------------------------------------------------------------|
| A governance | full / -Owner / -Trace / -Stop / all off                       | The three responsibility elements can be closed independently |
| B resilience | full / -circuit-breaking / -rollback / -reassignment / all off | The trio can be closed independently                          |
| C fault type | crash / drop_context / byzantine / duplicate / mixed           | Five semantic fault classes                                   |
| D fault rate | 0.05 / 0.1 / 0.2 / 0.33                                        | Fault injection ratio                                         |

Total cells =  $5 \times 5 \times 5 \times 4 = 500$ , with  $\geq 30$  seeds per cell (simulation cost is extremely low, so 100 seeds per cell is feasible directly). The main contrast is pre-registered as "full governance vs all off  $\times$  mixed faults."

**Table 2 Primary and guardrail metrics (pre-registered definitions)**

| Type      | Metric                    | Definition                                                  | Expected direction              |
|-----------|---------------------------|-------------------------------------------------------------|---------------------------------|
| Primary   | completion rate           | accepted/total (fraction of work orders accepted by QA)     | governance $\uparrow$           |
| Guardrail | rework count              | times a rework item is re-hung on the blackboard            | not excessively increased       |
| Guardrail | total messages            | total blackboard/dispatch/QA messages                       | not excessively increased       |
| Guardrail | false-isolation rate      | fraction of healthy nodes broken                            | the lower the better            |
| Guardrail | Trace breakage            | number of broken-link indices returned by hash-chain verify | the lower the better (0 best)   |
| Guardrail | spurious kill-switch rate | fraction globally stopped in recoverable scenarios          | the lower the better            |
| Process   | wall clock (CPU)          | CPU time per cell                                           | recorded, not a main conclusion |

#### 4.9 Quality Gates (ab-experiment-analysis)

- **SRM check:** The actual number of seeds per cell matches the allocation; fault-injection hit rate is registered.
- **No peeking, no sample deletion:** No selectively deleting seeds or changing definitions for significance.
- **Separating exploratory from confirmatory:** Subgroup analyses by domain and by topology are exploratory and reported separately from pre-registered main results.
- **Negative results are also published:** For example, an element having no significant benefit at low fault rate is itself a practical implication.

#### 4.10 Contrast with the Exemplar Paradigms (AlexNet ablation table / Kaplan conclusion map / pre-registration discipline)

The experiment design of this paper forms a methodological contrast with the three reverse-engineered exemplars:

- **Contrast with AlexNet's "numeric ablation table per trick":** AlexNet pairs dual-GPU, LRN, and overlapping pooling each with a two-column comparison of "how much error increases after

removal." This paper, dual to that, makes governance mechanisms into independently closable switches (–Owner / –Trace / –Stop / –circuit-breaking / –rollback / –reassignment), and the expected output of the full-factorial experiment is precisely a comparison table of "after removing a governance element, how much the completion rate drops and what mechanically detectable failure class appears" (Table 4 already gives the pilot's qualitative version of "fault type × relied-on mechanism × isolation method").

- **Contrast with Kaplan's "conclusion map + actively seeking contradictions":** Kaplan's introduction first states the conclusion and then exposes its own failure intersections. This paper, dual to that, actively reports "honest failure (kill-switch)" as a first-class result ( § 6.4)—this is not demonstrating failure, but explicitly measuring the contradiction that "recovery capacity has an upper bound" in a causal experiment.
  - **Pre-registration discipline (specific to this paper):** The primary metric (completion rate), guardrails (reworks/messages/false isolation/trace breakage), ITT stopping rules, and seed table are all frozen before running the full factorial ( § 4.4, § 4.9), eliminating "picking metrics after seeing results," the most common validity threat in systems research; the current single-seed pilot is the first step of "design first, data later," not a causal conclusion.
- 

## 5 Experimental Setup

### 5.1 Data Source and Configuration

- Sole data source: `reports7/matrix_scale_demo.json` ( `evidence_grade: cpu-proto` ), generated by `scripts7/matrix_demo.py`.
- Clean run: `run_mission(standard_workload(24, seed=11), MatrixConfig(), seed=1)`.
- Fault-injection run: same work orders, injecting `faults = {"spec-kinematics-0": "crash", "spec-spatial-1": "drop_context", "spec-dataflywheel-2": "byzantine"}`, `seed=1`.
- Byzantine QA: `qa_f=2`, i.e. the first 2 QA vote against acceptance.

### 5.2 Hardware Fingerprint and Evidence Grade

- Linux CPU; Python 3.12.11; PyTorch 2.14.0+cpu; 2 threads; device=cpu.
- Report sha256: `reports7/matrix_scale_demo.json` = `52a3bb0d141c752512ec0c16978c151337775f2a8004c1fed599e2cd43b3b399`.
- Evidence grade: this pilot is `cpu-proto`; after the full-factorial multi-seed completion, the main results will report `verified` (fixed seeds can be re-run).

### 5.3 Single-Seed Limitations

The current report is a single-seed (`seed=1`) pilot point estimate; the full-factorial RCT ( $5 \times 5 \times 5 \times 4$ ) and  $\geq 30$  seeds are pending. All "completion rates" in this paper are cell values under that seed, with no population inference; the statistics (OR, CI,  $p$ ,  $\eta^2$ ) are left for the re-run and are not fabricated here.

### 5.4 Randomization, Allocation, and Blinding

In the full-factorial experiment, the randomization unit is the independent seed: each (factor  $A \times B \times C \times D$ ) cell runs  $\geq 30$  seeds, and under each seed it is randomly decided which nodes are injected with faults (within the fault type and fault rate specified by factors C/D). The purpose is to keep "which nodes the faults land on" from being systematically correlated with "topology position," thereby

isolating the causal effect of governance mechanisms. Since this test bench is a deterministic CPU simulation with no human scoring, blinding is not done by blind human judgment but by "completion rate = mechanical verify," which guarantees objective measurement—there is no rater subjective preference. Inclusion/exclusion rules are pre-registered: once a seed starts, it is not removed because intermediate results are good or bad; cells where the kill-switch trips are retained as failures under ITT.

## 5.5 Relationship Between the Pilot and the Full Factorial

The relationship between the current report and the full-factorial design must be made clear: Sec. 6 reports the single-seed pilot result of the one cell "full governance × mixed faults (3 faulty specialists + 2 Byzantine QA)"; it is not the conclusion of the full-factorial RCT but a feasibility demonstration of the design—proving that this set of mechanisms can run through, isolate faults, and honestly fail. Only the full-factorial RCT answers "the marginal causal contribution of each mechanism." This paper states the pilot and the design separately precisely to avoid "passing off a single-point demo as a causal conclusion."

## 5.6 Why These Three Faults

Crash, drop\_context, and byzantine are chosen as pilot faults because they cover three typical positions in the multi-agent failure spectrum: crash is "no response at all" (corresponding to a microservice outage), drop\_context is "responds but did not receive necessary context" (corresponding to handoff failure, specific to Agent collaboration), and byzantine is "responds and looks normal but outputs an error" (the most dangerous, because the executor layer cannot detect it). Duplicate as a fourth class is included in the full factorial. This choice lets the pilot simultaneously test three different layers of failure—"can executor circuit breaking handle crash," "can rollback/reassignment handle context loss," and "can acceptance consensus handle Byzantine"—rather than testing only one.

---

# 6 Results

## 6.1 Clean-Run Baseline

With no faults, the matrix completes 24 of 24 work orders, with 24 QA rounds, 0 reworks, 48 blackboard messages, and Trace length 48; the market budget of 240 is fully paid and conserved, the six governance items (state\_loss / duplicate\_work / no\_closer / premature / unbounded / trace\_gaps) are all 0, and Trace verification passes (clean column of Table 3). This is the baseline of "governance working cleanly as designed."

The meaning of the clean baseline is that it establishes "governance should have no side effects when there are no faults." If the clean run itself produced reworks, duplicate work, or Trace breakage, that would indicate the governance mechanisms are over-sensitive or buggy; whereas the clean run's 0 reworks, all-zero six governance items, and intact Trace show that on the normal path governance is merely "standing by" and does not interfere with normal workflow. Only by comparing the clean baseline with fault injection can "the price governance pays for faults" be separated from "the inherent overhead of governance itself." This contrast is a prerequisite for causal inference: we need to know the difference under the same configuration between the treatment group (with faults) and the control group (without faults) in order to attribute it to fault governance.

## 6.2 Fault-Injection Pilot (Main Pilot Result)

After injecting 3 faulty specialists: 24/24 completion, 25 QA rounds, 5 reworks, 54 blackboard messages; all 3 faulty specialists (spec-kinematics-0, spec-spatial-1, spec-dataflywheel-2) enter `isolated_agents` (breaker isolation), with no group breaking and no kill-switch trip; the six governance items are all 0, the market is conserved, and the Trace length is 50 with verification passing.

**Table 3 Clean vs fault injection (24 work orders, cpu-proto, single seed=1)**

| Metric                       | Clean     | Fault injection |
|------------------------------|-----------|-----------------|
| accepted                     | 24/24     | 24/24           |
| QA rounds                    | 24        | 25              |
| rework retries               | 0         | 5               |
| blackboard messages          | 48        | 54              |
| isolated faulty specialists  | 0         | 3               |
| group breaking / kill-switch | none / no | none / no       |
| six governance items         | all 0     | all 0           |
| market conservation          | true      | true            |
| Trace integrity              | true(48)  | true(50)        |

See Figure 1 and Figure 2.

It is worth noting that fault injection did not pull the completion rate down from 24/24, but pushed QA rounds from 24 to 25 and reworks from 0 to 5. This shows that at this pilot's fault rate (3 faulty specialists, 2 Byzantine QA), the recovery capacity of the governance mechanisms has headroom—it uses one extra acceptance round and five reassignments to pull the contaminated work orders back to completion. But this is only an observation under a single seed and low fault rate; when factor D raises the fault rate to 0.33, or factor A turns off some governance element, the completion rate will very likely no longer stay at 100%, and that is exactly the "recovery-capacity boundary" the full-factorial experiment is meant to measure. This paper does not pretend on a single seed to know where that boundary is.

**Figure 1 Comparison of core metrics between the clean fleet and the fault-injected fleet (24 work orders, single seed=1).** Grouped bars: two bars per group, blue=clean fleet, orange=fault injection (3 faulty specialists: one each of crash/drop\_context/byzantine + 2 Byzantine QA); the vertical axis is the corresponding metric value, and groups from left to right are accepted (24 vs 24), QA rounds (24 vs 25), rework retries (0 vs 5), and blackboard messages board\_messages (48 vs 54). Both arms complete 24/24, with the difference on the cost side (1 extra QA round, 5 reworks, 6 blackboard messages). Values come one by one from `clean.*` and `stressed.*` in `reports7/matrix_scale_demo.json` (cpu-proto).

**Figure 2 Comparison of the six governance-audit items and settlement/Trace integrity (clean vs fault injection).** The horizontal axis is the governance/process metric and the vertical axis is the count (or boolean converted to a number): the six items `state_loss`, `duplicate_work`, `no_closer`, `premature_completion`, `unbounded`, and `trace_gaps` are all 0 in both arms; market conservation conserved (`budget=payout=240`) and Trace verification `trace_verified` are both true in both arms (Trace length 48 vs 50, the 2-record difference being rework items re-entering the chain). This is a contrast of "governance not breaking audit invariants": fault injection raises the cost-side metrics, but the six audit

items, conservation, and chain integrity are maintained. Source as above (`*.governance.*` / `*.market.conservd` / `trace_verified` in `matrix_scale_demo.json`, `cpu-proto`).

### 6.3 The Role of Each Governance Mechanism in the Pilot

Combined with the source code, we can interpret how the three fault classes are handled separately by governance:

- **crash (spec-kinematics-0)**: The specialist fails directly; `grid.record(..., False)` accumulates errors and, after the threshold is reached, OPEN isolates it; the in-flight work order's `rollback` truncates the half-written Trace, releases the blackboard, and reassigns to a healthy kinematics specialist.
- **drop\_context (spec-spatial-1)**: Execution fails with `missing_context`, likewise triggering breaker isolation and reassignment; the `state_loss` detector of `audit_run` records this class of failure (this pilot's governance `state_loss=0`, showing that the healthy node after reassignment completed the context and left no half-written state).
- **byzantine (spec-dataflywheel-2 + 2 Byzantine QA)**: The specialist outputs a wrong result (`true_result+7`); among the QA committee the honest votes still form a majority (7 people, 2 Byzantine,  $n \geq 3f+1$  holds), acceptance is judged reject, a quality failure is recorded, and the item is re-hung on the blackboard as rework; this byzantine specialist is broken and isolated through accumulated quality failures. This supports H3: byzantine is isolated through QA consensus plus accumulated quality failures, rather than through executor crash breaking alone.

We organize "fault type  $\times$  relied-on governance mechanism  $\times$  expected isolation method" into Table 4.

**Table 4 Correspondence between fault types and governance mechanisms (pilot observation + expectation)**

| Fault type                           | Failure manifestation         | Relied-on mechanism                         | Isolation/recovery method                                           | Pilot observation                        |
|--------------------------------------|-------------------------------|---------------------------------------------|---------------------------------------------------------------------|------------------------------------------|
| crash                                | no output                     | breaker+rollback+reassignment               | error accumulation<br>→ open isolation,<br>reassign to healthy node | spec-kinematics-0 isolated               |
| drop_context                         | missing_context               | rollback+reassignment                       | truncate half-written trace,<br>reassign to complete context        | spec-spatial-1 isolated                  |
| byzantine                            | looks successful but is wrong | QA consensus + accumulated quality failures | acceptance reject<br>→ rework → accumulated breaking                | spec-dataflywheel-2 isolated             |
| byzantine QA                         | votes against                 | BFT-lite quorum                             | honest majority wins, reject not overturned                         | 2 Byzantine QA did not affect acceptance |
| all-domain fault + zero retry budget | no node to reassign to        | kill-switch                                 | honestly stop, report non-completion                                | counterexample (qualitatively recorded)  |

This table shows a core judgment: **different faults need different governance mechanisms**. Crash can be handled by executor breaking, while byzantine must rely on acceptance-layer consensus. This is

exactly why this paper factorizes governance—one cannot use a single switch to broadly say "turning on governance is good," but must measure marginal contributions mechanism by mechanism.

## 6.4 Counterexample: Honest Failure (Kill-Switch)

This repo records a key counterexample: when **all-domain specialists fail** and the **retry budget is set to zero**, `run_mission` will, because `retries > global_retry_budget`, call `grid.kill.trip("retry_budget_exceeded")`, the loop stops, and the result is accepted`<24, governance.ok=false`. **The system honestly reports failure and does not fake success.** This is one of this paper's most important design principles: the purpose of governance is not "always showing success," but recovering when recovery is possible and clearly failing and leaving auditable evidence when it is not. The exact count of this counterexample is `[RESULT_NEEDED: usage=kill-switch scenario accepted count and governance item details]` (currently recorded qualitatively in the one-pager and CHANGELOG; the exact value requires re-running that configuration to confirm).

Why is this counterexample scientifically important? Because it reveals an often-overlooked boundary of governance systems: **recovery capacity has an upper bound, and beyond that bound one should stop rather than force through.** If the retry budget were infinite, the system would, in a scenario where all nodes are broken, keep reassigning, keep failing, and keep looping, neither converging nor stopping, ultimately producing untrustworthy results. The existence of the kill-switch is precisely to install a clear stop valve for this "infinite retry." In this paper's design, after the kill-switch trips the system enters an "honest failure" state: accepted is clearly less than the total, `governance.ok` is false, and the Trace completely records every failure and the stop reason. This is far friendlier to downstream systems than "faking completion"—downstream can alert and bring in human intervention accordingly, rather than being misled by an erroneous result.

## 6.5 Tests and Regression

Tests directly related to this paper: `tests7/test_v744_governance.py` (9 items), `test_v748_circuit_breaker.py` (11 items), `test_v745_decision.py` (12 items), `test_v750_matrix.py` (9 items); as of v7.5.0 the full regression of 236 items is all green.

## 6.6 Pilot Contrast Hypothesis by Hypothesis

Comparing the three hypotheses one by one with pilot observations, we can see which are preliminarily supported and which still await full-factorial testing.

- **For H1 (marginal governance elements):** In the pilot the six governance items are all 0, showing that under "full governance" failures such as state loss, duplicate work, and unclear responsibility do not appear. But what H1 asks is "what happens when an element is removed," which requires the "–Owner/–Trace/–Stop" cells in the full factorial to answer. The current pilot can only say "these failures do not appear under full governance," not "the marginal contribution of each element."
- **For H2 (trio recovery):** Both crash and drop\_context are isolated and reassigned under the trio, and the completion rate stays 24/24, preliminarily supporting H2. But H2 also includes the guardrail dimension of "too-low thresholds falsely isolating healthy nodes," which the single-point pilot at the fixed threshold (0.5) cannot measure; the full factorial needs to adjust the threshold.
- **For H3 (Byzantine requires consensus):** Under a byzantine specialist and 2 Byzantine QA, the honest majority still correctly rejects and accumulates to break the byzantine node, preliminarily supporting H3. The full factorial needs to increase the f ratio to test the boundary of consensus under harsher Byzantine proportions.

This "hypothesis—pilot—pending" contrast lets readers clearly know the current evidence strength of each conclusion rather than inflating a single-seed demo into a causal conclusion.

## 6.7 Conservation and Economic Guardrails

Like the clean run, the fault-injection run satisfies market conservation: budget 240, payout 240, slashing 0, balance sum 240, `conserved=true`. This means fault governance did not break the settlement invariant—even with 3 nodes isolated and 5 reworks, the system did not show "money spent on unfinished work orders" or "duplicate work paid repeatedly." In the full-factorial experiment, we plan to use "whether conservation is broken" as an additional process guardrail: when a governance switch is turned off (such as removing reassignment), will duplicate work be paid, or will Byzantine results be paid. This is where mechanism design intersects fault governance, and this paper currently only reports the point "conservation holds under full governance."

## 6.8 Trace Integrity Evidence

The clean run Trace length is 48 and the fault run 50, with both runs' `trace_verified` being true. The extra 2 records correspond to rework items re-entering the chain. The key observation is that the rollback action did not break the hash chain—this proves the `snapshot/rollback` design is correct (truncating to the snapshot point and relinking subsequent records). In the full factorial, the "remove rollback" cell is expected to observe nonzero `trace_gaps` or residual half-written state, which is precisely the measurable expectation of "what happens when Trace/rollback is removed" in H1.

---

# 7 Discussion and Threats to Validity

## 7.1 Construct Validity

Completion rate is "the fraction of work orders ultimately accepted by the QA committee," mechanically decidable (result equals ground truth and domain matches), involving no subjective scoring. Faults are manually injected discrete types and are not equal to the real LLM fault distribution. Messages/ticks are not equal to wall clock.

To elaborate further, the primary metric is chosen as completion rate rather than "end-to-end latency" or "human satisfaction" because the former is mechanically decidable and reproducible on this test bench, while the latter two require real LLMs or human scoring beyond the current CPU gate. This is a deliberate trade-off: trading a mechanically decidable primary metric for the credibility of causal inference, rather than trading vague satisfaction for superficial richness. The guardrail metrics (reworks, messages, false isolation, trace breakage) then complete the characterization of "the cost behind completion rate," avoiding the misjudgment of looking only at a single primary metric.

## 7.2 External Validity

Agents are deterministic processors, not real LLMs. The nondeterministic failures, long-tail latency, and prompt failures of real LLMs are not covered. Conclusions are limited to protocol-layer causal effects; upgrading to real multi-LLM requires an API-key gate.

## 7.3 Internal Validity and the Single Seed

The current report is a single-seed pilot, and 24/24 with 5 reworks are point estimates. The causal effects of the full-factorial RCT need  $\geq 30$  seeds and a logistic mixed model to estimate; guardrails such

as false-isolation rate and trace-break rate are 0 in the current pilot but may be nonzero when the fault rate rises, and are pending measurement. This paper does not fabricate these statistics.

## 7.4 The Value of the Counterexample

"Honest failure" is avoided as a "failure case" in most fault-governance papers, but in engineering it is as important as "successful recovery": it defines the honest boundary of the governance system. This paper reports it as a first-class result, a deliberate methodological choice.

## 7.5 The Cost of Governance: It Is Not Free

The pilot data already hint that governance is not free: fault injection has, compared with the clean run, 5 extra reworks, 6 extra blackboard messages, 2 extra Trace records, and 1 extra QA round. These "extra overheads" are exchanged for localized faults. But the cost goes further—in the full-factorial design we must also measure two costs of "over-governance": the first is **false isolation**, where when the breaker threshold is over-sensitive (such as `threshold=0.5`, `min_requests=2`), a healthy node with an occasional failure may be wrongly isolated, leading to fewer executable work orders; the second is **excessive rework**, where when the QA committee is too conservative, results that could have been accepted are repeatedly rejected, producing meaningless rework. These two costs cannot be measured under the current single seed and single threshold, and are exactly what the full-factorial RCT must answer: whether the rise in completion rate comes at the cost of degraded false-isolation rate and rework count.

## 7.6 Why Byzantine Isolation Cannot Rely Only on Executor Breaking

The deep meaning of H3 is worth expanding. A byzantine specialist returns wrong results "looking successful" every time, and the executor-layer breaker only looks at "whether an error is reported," so it is completely insensitive to it—it never crashes and never has `missing_context`. If relying only on executor breaking, the byzantine node would keep being dispatched, keep returning errors, and keep contaminating results. This system's solution is to use the "acceptance predicate" as the failure signal: the QA committee votes on the result, and a reject records a quality failure, accumulating to the threshold before breaking that node. This extends the definition of "failure" from "execution exception" to "acceptance not passed," which is the key by which multi-agent governance differs from traditional microservice governance. BFT-lite consensus further guarantees that even if there are Byzantine members within the QA committee, the honest majority can still correctly reject and will not be captured into a "false acceptance pass."

## 7.7 Relationship with P1 and P12

This paper is one of the three matrix P0 papers. P1 provides the "topology and scale" dimension: the hierarchical tree determines fan-in and rounds, while this paper overlays fault governance on such a hierarchical matrix and measures completion rate. P12 provides the "evidence grading" dimension: this paper's single-seed pilot, pending full factorial, and honest recording of the kill-switch counterexample are precisely a practice of the evidence discipline P12 advocates. The three do not overlap: P1 asks about "structural cost," P3 asks about "governance causal effect," and P12 asks about "how the method is honestly executed."

## 7.8 Internal Validity Threats

Even after the full factorial is run, several internal validity threats remain to be watched. The first is **the coupling of the fault model and governance switches**: the faults in this paper are manually injected discrete types, and if real LLM faults are continuous, correlated, and long-tail, the extrapolability of the

factor experiment is limited. The second is **inter-seed independence**: if multiple work orders under the same seed share random state, inter-seed variance may be underestimated; the full-factorial experiment needs to check within-seed/between-seed variance components. The third is **multiple comparisons**: 500 cells  $\times$  multiple metrics must use Holm or family-wise error-rate control, otherwise there is a risk of "picking significant cells." The fourth is **the implementation correctness of governance switches**: whether removing an element is truly equivalent to "that mechanism not taking effect" needs unit tests to guarantee switches have no side effects—this repo's `test_v744 / test_v748` already guard each mechanism with 9+11 tests.

## 7.9 External Validity and Generalizability

This paper's conclusions are limited to the scope of "deterministic CPU test bench, manually injected discrete faults, mechanically decidable work orders." Its generalizability to real production systems depends on: whether the real fault distribution overlaps with this paper's four fault classes, whether real Agent outputs can be judged by the acceptance predicate, and whether real networks introduce latency and packet loss this paper does not model. All of these need to be tested by re-running with real multi-LLM after the gate is unlocked. This paper does not claim protocol-layer conclusions can be directly generalized into production SLAs.

### 7.10 Review of the Falsifiability of Conclusions

Returning to Sec. 3, this paper's three hypotheses are all falsifiable: if the full-factorial experiment finds the completion rate unchanged after removing a governance element, H1 is weakened; if crash still cannot complete under the trio, H2 is overturned; if byzantine can be executor-broken without consensus, H3 is weakened. This practice of "placing conclusions where they can be negated by experiment" is the key by which causal research differs from demos.

---

## 8 Resource Gates and Applicability Boundaries

- **Achieved**: The main governance mechanisms, matrix loop, and single-seed pilot, reproducible on pure CPU.
- **Pending (completable on CPU)**: The full-factorial RCT ( $5 \times 5 \times 5 \times 4$ )  $\times \geq 30$  seeds, power analysis, and CIs.
- **Gates not reached**: Real multi-LLM fault distribution (API key), real network faults (multi-machine cluster).

### 8.1 Experiment Cost and Feasibility

The full-factorial experiment has 500 cells, 30 seeds per cell = 15,000 missions. Since a single mission is millisecond-level on CPU (deterministic, no LLM calls), running all of it is extremely low-cost in a pure CPU environment, which is why this paper dares to write the full-factorial design into pre-registration rather than leaving it at the concept level. By contrast, if every mission had to call a real LLM API, the cost would scale linearly with the number of seeds and the full-factorial design would be economically infeasible—this is also the strategic reason this repo chooses "first doing the causal design solidly on a deterministic CPU test bench, and then connecting real LLMs after the gate is unlocked."

## 8.2 When Conclusions Can Be Upgraded

This paper's current conclusions (cpu-proto, single-seed pilot) can be upgraded after the following conditions are met: (1) the full factorial  $\geq 30$  seeds is run and the primary metric completion rate gives OR and 95% CI; (2) the guardrails false-isolation rate and trace-break rate are actually measured when the fault rate rises; (3) after the real multi-LLM gate is unlocked, re-running on the same test bench with real workers to test whether the protocol conclusions hold under real nondeterminism. Before then, this paper only claims "the mechanisms work as designed under the deterministic pilot," not "they necessarily work in production."

## 8.3 Operational Advice for Engineering Teams

Based on this paper's design and pilot, there are three actionable pieces of advice for engineering teams building multi-agent systems: first, extend the definition of fault from "call failure" to "acceptance not passed," otherwise byzantine-class faults will be completely missed by executor breaking; second, set a clear upper bound on recovery capacity (retry budget + kill-switch) and honestly report failure when the bound is reached, rather than retrying infinitely or forcing success; third, make governance mechanisms into independently switchable components, so the marginal contribution of each mechanism can be separated in experiments rather than broadly trusting "turning on governance is good." All three pieces of advice come from mechanisms this paper has implemented and tested, not from unsupported claims.

---

## 9 Conclusion

This paper factorizes and pre-registers multi-agent fault governance: the trio (circuit breaking/rollback/reassignment) and responsibility governance (Owner/Trace/Stop) keep the completion rate under the three fault classes at 24/24 in the single-seed pilot and isolate all faulty nodes, while the system honestly trips the kill-switch to report failure in unrecoverable scenarios. The primary metric (completion rate), guardrails (reworks/messages/false isolation/trace breakage), and stopping rules are pre-registered, and the full-factorial multi-seed experiment is pending before submission. This paper's stance is: the scientific goal of fault governance is not "eternal success," but "causally knowing which mechanisms work, at what cost, and when one should honestly fail."

Looking ahead, the full-factorial experiment will answer questions the current pilot cannot: removing any of Owner/Trace/Stop, how much does the completion rate drop and what mechanically detectable failure class appears? Adjusting the breaker threshold from 0.3 to 0.7, what does the trade-off curve between false-isolation rate and completion rate look like? Can byzantine faults still be isolated when QA consensus is weakened (f increased)? The answers will upgrade this paper from a "mechanism demo" to a "causal-effect measurement." But whatever the results, the pre-registration discipline and honest-failure principle this paper insists on will be retained: no peeking, no sample deletion, and no passing off a single-seed point estimate as a population conclusion.

Finally, this paper hopes to convey to the multi-agent engineering community a simple but often-overlooked judgment: the value of fault governance lies not in making the system look perpetually healthy, but in, when the system breaks, **having the ability to localize, having the ability to recover, and having the ability to admit it cannot recover.** All three need to be designed, measured, and pre-registered for verification, rather than relying on a black-box narrative of "the agent will fix itself." What this paper delivers is precisely such an experimental framework and single-seed pilot evidence that is runnable, auditable, and falsifiable under pure CPU, reproducible conditions.

---

## References

### A. Verified Literature (verified via the "Master Reference Library" on 2026-09-19, fields copied from the master library)

1. **Castro, M. & Liskov, B.** (1999). Practical Byzantine Fault Tolerance. Proceedings of OSDI 1999 (USENIX OSDI'99). ( $n \geq 3f+1$ ,  $2f+1$  quorum, view change)
2. **Angelopoulos, A. N. & Bates, S.** (2021). A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511. (statistical foundation for degrading/handoff when uncertainty is high)
3. **Ahn, M., Brohan, A., Brown, N. et al.** (2022). Do As I Can, Not As I Say: Grounding Language in Robotic Affordances (SayCan). arXiv:2204.01691. (switching person/skill when affordance is not satisfied)
4. **Zheng, L., Chiang, W.-L., Sheng, Y. et al.** (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. Proceedings of NeurIPS 2023 Datasets and Benchmarks. arXiv:2306.05685. (agreement rate and biases of multiple QA judges)
5. **Wu, Q., Bansal, G., Zhang, J. et al.** (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155. (multi-agent dead-loop/cost-inflating failure mode)
6. **DeepSeek-AI** (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948. (recovery/self-play driven by verifiable rewards after failure)

### B. Pending Verification Literature (not included or verified in the master library, placeholders retained, no completion from memory)

1. [CITATION NEEDED: circuit breaker pattern microservices fault tolerance] — candidate: the circuit-breaker pattern and microservice fault tolerance (the master library currently has no verified entry).
2. [CITATION NEEDED: hash chained audit log ownership attribution] — candidate: hash-chained audit logs and ownership registration.
3. [CITATION NEEDED: chaos engineering LLM agents fault injection controlled experiment] — candidate: chaos engineering/fault injection and controlled experiments.
4. [CITATION NEEDED: pre-registration empirical software engineering experiment reproducibility] — candidate: pre-registration and empirical software-engineering reproducibility.

---

## Appendix A Reproduction Commands

```
cd release-v5.4.4/udox-engine && git checkout v7.5.0 # 1a71270
python scripts7/matrix_demo.py # generate matrix_scale_demo.json
pytest tests7/test_v744_governance.py tests7/test_v748_circuit_breaker.py \
    tests7/test_v745_decision.py tests7/test_v750_matrix.py -q
```

## Appendix B Test Index

- `test_v744_governance.py`: TraceLedger hash chain, audit\_run five failure classes, StopCondition.
- `test_v748_circuit_breaker.py`: three-tier breaking, snapshot/rollback, reassign.

- `test_v745_decision.py`: topology decision tree, high-risk escalation.
- `test_v750_matrix.py`: matrix end-to-end, fault completion, conservation, Trace.

## Appendix C Evidence Ledger (spot-check of 5 traceable numbers)

| # | Number                                        | Source                 | Field                                                         |
|---|-----------------------------------------------|------------------------|---------------------------------------------------------------|
| 1 | clean completion 24/24, QA 24, reworks 0      | matrix_scale_demo.json | clean.accepted / qa_rounds / retries                          |
| 2 | fault completion 24/24, reworks 5             | matrix_scale_demo.json | stressed.accepted / retries                                   |
| 3 | isolating 3 faulty specialists                | matrix_scale_demo.json | stressed.isolated_agents                                      |
| 4 | six governance items all 0, conservation true | matrix_scale_demo.json | stressed.governance.* / stressed.market.conservated           |
| 5 | breaker threshold/minimum sample/QA count     | source MatrixConfig    | breaker_threshold=0.5 / min_requests=2 / qa_voters=7 / qa_f=2 |

## Appendix D Author's Intended-Use Statement

- **Target journals/conferences:** candidates include the AI engineering track of ICSE/FSE/ASE, ISSRE, AAMAS, the ESEC/FSE industry track; journals TSE, JSS, Empirical Software Engineering. Quartiles/ IF/deadlines [to be checked].
- **Pre-registration plan:** The Sec. 4.4 full-factorial design, primary metric completion rate, guardrails, and ITT stopping rules are frozen before running data; the seed table and analysis scripts are made public with the code.
- **Data and code availability:** Apache-2.0; tag v7.5.0 (commit 1a71270); report sha in Appendix A.
- **AI-use statement:** The first draft was AI-assisted, and the authors are responsible for checking the numbers and scientific meaning; external figures are all marked unverified.

## Appendix E Internal Review Record (five-dimensional reviewer self-assessment, look-only)

1. **Novelty (7/10):** Circuit breaking/BFT are not new; the combination of "factorized governance switches + pre-registered RCT + honest failure as a first-class result" adds value.
2. **Rigor (7/10):** Mechanisms and numbers are traceable; deductions for the single seed and unrun full factorial.
3. **Evidence strength (5/10, expected 8/10 after the full factorial):** Currently only one single-seed pilot cell.
4. **Relevance (8/10):** Fault governance is a core problem of Agent productionization.
5. **Writing (8/10):** IMRaD + complete pre-registration, with the counterexample honestly reported. - **Biggest hard flaw:** The full-factorial RCT has not yet been executed; the current state is "design + single-cell pilot," and the full-factorial multi-seed must be completed before submission. - **Must-do before submission:** (a) run the full factorial  $5 \times 5 \times 5 \times 4 \times \geq 30$  seeds; (b) estimate false-isolation rate and trace-break rate; (c) make the kill-switch counterexample count precise.

## Appendix F Design Principles and Term Summary

### F.1 Three Design Principles

Three principles with universal meaning for multi-agent production systems can be extracted from this paper's fault-governance design:

1. **Failures must be isolated in layers:** executor crashes use breaking, semantic errors use acceptance consensus, and globally unrecoverable situations use the kill-switch; different faults use different governance layers, with no reliance on a single switch.
2. **Leave no half-written state:** on failure, first snapshot and roll back the Trace, then reassign, ensuring auditability is not broken by recovery actions.
3. **Honest failure is better than false success:** when recovery capacity is exhausted, the system should clearly stop and report non-completion rather than forcing success; honest failure is safer than erroneous success.

### F.2 Term Correspondence

- **Completion:** The work order is ultimately accepted (stop) by the QA committee.
- **Rework:** Acceptance not passed, and the work order is re-hung on the blackboard as a rework item.
- **Isolation:** The specialist enters the breaker open state and is no longer dispatched.
- **Reassignment:** A failed work order is transferred from a faulty node to a healthy node.
- **kill-switch:** The global stop valve; once set, all dispatch stops immediately.
- **ITT:** intention-to-treat, analyzing by the initial allocation in a randomized controlled experiment, not removing based on result quality.

### F.3 One-Sentence Summary

The scientific goal of multi-agent fault governance is not to train the system to "never fail," but to turn "which mechanisms can causally recover completion, at what cost, and when one should honestly stop" into a layered, switchable, pre-registered, reproducible measurement; on a pure CPU deterministic test bench, this paper delivers this design together with the single-seed pilot and the honest-failure counterexample, and takes the full-factorial multi-seed experiment as a clear next step.

### F.4 Expected Dialogue with Related Fault-Governance Literature

On submission, this paper expects to dialogue with three classes of literature: first, the microservice circuit-breaking and chaos-engineering literature, where this paper will point out they mostly inject faults at the infrastructure layer while this paper additionally injects semantic faults; second, the BFT consensus literature, where this paper will point out it does not improve the consensus algorithm itself but uses mature consensus as an acceptance-layer component, measuring its governance effect in Agent semantic acceptance; third, the multi-agent systems-engineering literature, where this paper will argue governance mechanisms should be factorized and pre-registered for measurement rather than reporting "success-rate improvement" as a whole. These three dialogue stances need refinement after literature retrieval and are currently only expected directions.

### F.5 Echo of This Repo's Evidence Discipline

In writing, this paper strictly executes this repo's evidence discipline: all numbers come from `reports7/matrix_scale_demo.json` and are marked `cpu-proto`; the single-seed pilot does not fake a population

conclusion; the full-factorial multi-seed is marked pending; the kill-switch counterexample is honestly recorded; external industry figures are marked unverified and not cross-validated with this repo's numbers. This discipline is not decoration but a prerequisite for the credibility of this paper's causal conclusions—if a fault-governance paper itself does not honestly report the "cannot fix" scenario, it has no standing to claim "honest failure" is the goal of system design. This paper also places itself under this standard for examination.

## F.6 Memo on Trial Configuration and Parameter Sources

All parameters involved in this paper can be located in the source code: `MatrixConfig` (`udos7/topology/matrix.py`) gives `specialists_per_domain=3`, `qa_voters=7`, `qa_f=2`, `reward_per_order=10`, `max_rounds=20`, `breaker_min_requests=2`, `breaker_threshold=0.5`, `global_retry_budget=999`; `AgentBreaker` (`circuit_breaker.py`) gives `cooldown_ticks=3` and the closed/open/half\_open state machine; `StopConsensus` (`consensus.py`) gives  $n \geq 3f+1$  and the  $2f+1$  quorum. Fault-injection positions are specified by the `faults` dictionary in `scripts7/matrix_demo.py`. On submission reproduction, these parameters should be made public with the code, ensuring "where the numbers come from" is fully traceable.

## F.7 One-Sentence Restatement of the Main Result

On the single-seed pilot of 24 work orders, 9 specialists, and 7 QA (including 2 Byzantine), after injecting crash/drop\_context/byzantine faults, the hierarchical matrix, together with three-tier breaking, snapshot rollback, reassignment, and BFT-lite acceptance consensus, still achieves 24/24 completion, isolates all 3 faulty nodes, and incurs 5 reworks, with the six governance items, conservation, and hash-chained Trace integrity maintained; under the extreme scenario of all-domain faults plus zero retry budget, the system honestly trips the kill-switch to report non-completion. This is single-seed pilot evidence with design first and full-factorial multi-seed pending, not a population causal conclusion.

## F.8 Claims Not Made

To avoid over-claiming, this paper explicitly does not make the following claims: it does not claim "multi-agent fault governance necessarily works in production"; it does not claim "the 24/24 completion rate can be generalized to any fault rate"; it does not claim "the breaker threshold 0.5 is optimal"; it does not claim "the boundary under Byzantine proportions has been measured." All of these can only be answered after the full-factorial multi-seed and the real-LLM gate are unlocked. This paper currently only claims: on the deterministic CPU test bench, this set of factorized governance mechanisms works as designed and honestly fails when recovery is impossible.

## F.9 Acknowledgments and Limitation Statement

This paper's test bench is built on the accumulation of several earlier versions of this repo (three topologies, blackboard collaboration, BFT consensus, internal market, observability). The authors thank these early versions for providing runnable components. At the same time the limitation must be restated: the author is the developer, and confirmation bias exists; this paper hedges this bias by making all archives public, retaining the kill-switch counterexample, and labeling the single seed. Readers should treat this paper as an engineering report of "design + single-seed pilot," not as a completed causal conclusion.

## F.10 Closing

Fault governance does not make the system incapable of breaking, but makes the system, when it breaks, have rules to follow, evidence to consult, and a limit to stop at. This paper builds this "rules,

evidence, limit" into runnable code and reproducible experiments, and honestly reports how far it can currently go. This paper believes this honesty itself is the proper starting point of fault-governance research. After the next full-factorial multi-seed experiment is complete, this paper will replace the current single-seed point estimate with OR and confidence intervals and update the evidence-strength grades of all the above conclusions accordingly. Before then, this paper retains its positioning as a design and pilot report.